

반응형 포트폴리오 코드 완전 분석

순수 HTML · CSS · JavaScript로 만든 웹사이트의
모든 줄을 설명할 수 있게 되기 위한 학습 문서

작성자	김동석 (숭실대학교 AI소프트웨어학부)
저장소	github.com/dnsxkm/portfolio
배포	dnsxkm.github.io/portfolio
구성	index.html (202줄) · style.css (689줄) · main.js (306줄)

이 문서를 쓰는 법

이 과제의 최종 평가 기준은 “동작하는가”가 아니라 **“설명할 수 있는가”**입니다. 그래서 이 문서는 코드를 나열하지 않고, **각 줄이 왜 그 자리에 있는지를** 중심으로 씁니다.

① 2장(핵심 원리)을 먼저 읽고 → ② 3~5장을 코드 파일과 나란히 놓고 대조하며 읽고 → ③ 7장 예상 질문에 **문서를 덮고** 답해 보세요. 막히는 항목이 곧 다시 읽어야 할 장입니다.

목차

- 1. 1** 전체 구조 — 파일 3개의 역할 분담
브라우저가 이 사이트를 그리는 순서와 각 파일이 맡은 일
- 2. 2** 핵심 원리 — 이벤트 → 상태 → 렌더링
이 프로젝트 전체를 관통하는 단 하나의 사고방식 (가장 중요)
- 3. 3** HTML — 시맨틱 구조와 접근성
태그 선택 기준, alt·label·aria, head 인라인 스크립트, defer
- 4. 4** CSS — 토큰 · 레이아웃 · 반응형
CSS 변수, 다크모드, Flexbox vs Grid, 모바일 퍼스트, 상태 클래스
- 5. 5** JavaScript — 9개 블록 정독
DOM 선택부터 API 연동·폼 검증까지 한 블록씩
- 6. 6** ES6+ 문법 총정리
화살표 함수 · 구조분해 · 템플릿 리터럴 · 배열 메서드
- 7. 7** 예상 질문 22개와 모범 답안
발표·면접에서 실제로 나올 질문
- 8. 8** 요구사항 대조표 & 자가 점검
과제 요구사항 1~10을 코드 위치와 1:1로 매칭

표기 규칙

○ / ✕ 로 끝나는 문장은 그대로 시험·발표에 나올 수 있는 결론입니다. 파란 박스 는 반드시 이해해야 하는 핵심, 빨간 박스 는 실제로 이 프로젝트에서 겪은 함정입니다.

1. 전체 구조 — 파일 3개의 역할 분담

이 사이트는 파일 세 개로 되어 있습니다. 이 셋의 **경계가 어디인지**를 아는 것이 전부입니다.

파일	맡은 일	구체적으로
<code>index.html</code> 202줄	무엇이 있는가 (구조·의미)	섹션 6개, 폼, 버튼이 존재한다 는 사실. 내용물과 의미(시맨틱). 색이나 위치는 여기 없음
<code>css/style.css</code> 689줄	어떻게 보이는가 (표현)	색·크기·배치·애니메이션. " 어떤 상태일 때 어떻게 보일지 "까지 전부 미리 정의
<code>js/main.js</code> 306줄	언제 무엇이 바뀌는가 (동작)	사용자 행동을 감지해 클래스와 속성만 바꾼다 . 색을 직접 칠하지 않음

브라우저가 페이지를 여는 순서

1. `index.html` 을 위에서 아래로 읽기 시작
2. `<link rel="stylesheet">` 발견 → `style.css` 다운로드 시작 (파싱은 계속)
3. `<script>` (head 인라인) 발견 → 즉시 실행 = 저장된 테마를 `<html>`에 씌움 ★FOUC 방지
4. `<body>` 이하 모든 태그를 읽어 DOM 트리 완성
5. `<script src="js/main.js" defer>` → DOM 완성 후 실행
6. `main.js` 가 이벤트 리스너를 걸고, GitHub API를 호출
7. 사용자가 클릭/스크롤 → JS가 클래스를 바꿈 → CSS가 화면을 다시 그림

3번이 왜 특별한가

`main.js` 는 `defer` 때문에 4번이 끝나야 실행됩니다. 다크모드 복원을 거기서 하면 **흰 화면이 한 번 번쩍인 뒤** 어두워집니다 (FOUC, Flash of Unstyled Content). 그래서 그 5줄만 `<head>` 안에 인라인으로 두어, CSS가 처음 그려지기 **전에** 테마를 확정합니다.

폴더 구조와 그 이유

```
portfolio/
├── index.html      ← 진입점. GitHub Pages는 이 이름을 찾는다
├── css/style.css   ← 역할별 폴더 분리 (요구사항 1)
├── js/main.js
├── images/        ← 프로필·앱 스크린샷
└── README.md
```

왜 한 파일에 몰아넣지 않았나: ① HTML이 바뀌어도 CSS·JS는 브라우저 캐시에 남아 재다운로드가 없다. ② 파일별로 담당이 갈려 협업 시 충돌이 줄어든다. ③ 과제 요구사항 1번 항목.

경로에 관한 결론

자산 경로를 `css/style.css` 처럼 **상대경로**로 썼습니다. GitHub Pages는 `/portfolio/` 하위 경로에 배포되므로, `/css/style.css` 처럼 **절대경로**를 쓰면 도메인 최상단을 찾아가 CSS·JS가 전부 404가 납니다.

결론: 하위 경로 배포에서는 상대경로를 써야 한다 → [O](#)

2. 핵심 원리 — 이벤트 → 상태 → 렌더링

이 문서에서 딱 하나만 외운다면

JavaScript는 클래스와 속성만 바꾼다. 어떻게 보일지는 CSS가 전담한다.

다크모드를 예로 보겠습니다. JS가 하는 일은 글자 한 줄입니다.

```
// main.js — JS가 하는 일의 전부
root.setAttribute('data-theme', 'dark'); // <html data-theme="dark">
```

색을 정하는 것은 CSS입니다. JS는 색이 무엇인지조차 모릅니다.

```
/* style.css — 실제로 화면을 바꾸는 쪽 */
:root { --color-bg: #ffffff; --color-text: #1a1a1a; }
[data-theme="dark"] { --color-bg: #121212; --color-text: #e8e8e8; }

body { background: var(--color-bg); color: var(--color-text); }
```

[사용자]	[JS]	[DOM = 상태]	[CSS]
클릭	→ 속성/클래스 변경	→ data-theme="dark"	→ 변수 교체 → 전체 화면 갱신
		.nav-menu.active	
		header.scrolled	
		.to-top.show	
		.reveal.visible	
		input.invalid	

이 프로젝트의 5가지 상태 흐름 (요구사항 9번)

#	이벤트	바뀌는 상태	화면 변화
1	테마 버튼 클릭	<html data-theme> 값	CSS 변수가 통째로 교체 → 배경·글자·카드·폼·푸터 전부 반전
2	페이지 진입 / 재시도 클릭	로딩 → 성공 / 에러 / 빈 데이터	Projects 영역이 스피너 ↔ 카드 목록 ↔ 에러 박스로 교체
3	폼 입력 · 제출	필드별 유효/무효	에러 문구 표시·숨김, 입력칸에 .invalid 부착 → 테두리 빨강
4	스크롤	임계값(60px·300px) 초과 여부	header.scrolled, .to-top.show 부착/제거
5	요소가 화면에 진입	노출 여부	.reveal → .reveal.visible 로 페이드 인

React와의 연결 (다음 미션의 핵심)

React의 `useState` + 렌더링도 정확히 같은 분업입니다. **상태를 바꾸면 화면은 알아서 따라온다**가 공통 원리이고, React는 “상태가 바뀌면 어느 DOM을 고칠지”를 자동으로 계산해 줄 뿐입니다. 지금 손으로 하는 `classList.toggle` 이 그 자동화의 원형입니다.

왜 이렇게 나누는 게 이득인가

JS가 색을 직접 칠한다면	CSS에 맡기면 (지금 방식)
<code>el.style.background = '#121212'</code> 를 요소마다 반복 → 요소가 40개면 40줄	속성 1개만 바꾸면 CSS 변수 상속으로 전체가 따라옴
인라인 스타일이 생겨 과제 제약 위반	인라인 스타일 0개
색을 바꾸려면 JS를 고쳐야 함	디자인 변경은 CSS만 고치면 됨
<code>transition</code> (부드러운 전환)을 직접 구현해야 함	CSS <code>transition</code> 이 알아서 처리

3. HTML — 시맨틱 구조와 접근성

3-1. 왜 div가 아니라 시맨틱 태그인가

`<div>` 는 브라우저에게 **아무 의미도 주지 않는** 상자입니다. 시맨틱 태그는 의미를 줍니다.

	<code><div class="header"></code>	<code><header></code>
스크린리더	그냥 지나감	“배너 영역”으로 안내, 바로 점프 가능
검색엔진	추측해야 함	문서 구조를 파악
개발자	class 이름을 봐야 함	태그만 봐도 됨

랜드마크(landmark)는 시맨틱 태그 중에서도 특별한 지위로, 시각장애 사용자가 목록을 띄워 원하는 영역으로 바로 이동할 수 있게 합니다. `header` · `nav` · `main` · `footer` 와, **제목이 붙은** `section` 이 여기 해당합니다.

주의

`<section>` 은 **제목(`h2` 등)이 없으면 랜드마크로 인정되지 않습니다.** 이 프로젝트의 모든 `section` 에 `h2` 가 붙어 있는 것은 그 때문입니다(Hero는 `h1`).

3-2. section / article / div 판정 기준

태그	쓰는 기준	이 프로젝트에서
<code>section</code>	제목을 붙일 수 있는 주제 단위 묶음	About, Skills, Featured, Projects, Contact
<code>article</code>	떼어내도 혼자 말이 되는 독립 콘텐츠	GitHub 저장소 카드 1개, 코인 레벨 카드
<code>div</code>	의미 없음. 순수 스타일링용 래퍼	<code>.hero-actions</code> , <code>.about-inner</code> , <code>.projects-grid</code> , <code>.form-field</code>
<code>nav</code>	주요 내비게이션 링크 묶음	상단 메뉴 (푸터의 소셜 링크는 주요 내비가 아니라 <code>ul</code>)
<code>main</code>	페이지의 핵심 콘텐츠. 문서당 1개	header·footer를 제외한 전부

한 문장 판정법

“이 덩어리를 RSS 피드에 하나만 떼서 넣어도 말이 되는가?”

→ 된다 = `article` (저장소 카드: 이름·설명·별 개수 → 말이 됨)

→ 안 된다 = `section` (Skills: “기술 스택”이라는 제목이 있어야 의미가 생김)

3-3. 세 가지가 각각 맡는 일

수단	답하는 질문	누가 읽는가
태그 <code>section</code>	이게 무슨 종류인가	브라우저 · 스크린리더 · 검색엔진
<code>id</code> <code>about</code>	이게 어느 것인가 (문서에서 유일)	앵커 링크(<code>href="#about"</code>), JavaScript
<code>class</code> <code>btn</code>	이걸 어떻게 꾸밀지 (여러 개 가능)	CSS, JavaScript

실제로 겪은 실수 ①

처음에 `<hero>`, `<about>` 같은 태그를 직접 만들어 썼습니다. **HTML 태그는 표준에 정해진 목록만 존재하며**, 없는 이름을 쓰면 에러 없이 `HTMLUnknownElement` (기본 `display: inline`)로 처리되어 `width · height` 도 안 먹고 의미 전달도 0입니다.

결론: 섹션 이름은 태그가 아니라 `id` 로 붙인다 → **O**

3-4. 접근성 — 세 가지 장치

① alt — “무엇이 보이는지”를 쓴다

나쁨	좋음
<code>alt="이미지" / alt="profile.jpg"</code>	<code>alt="김동석 프로필 사진"</code>
파일명·형식은 정보가 아님	<code>alt="비트코인 일봉 차트. 지지선과 저항선이 점선으로 표시되고 각 선의 접촉 횟수가 함께 나타나 있다."</code>

예외: 순수 장식용 이미지는 `alt=""` 로 **비워두는 것이 정답**입니다 — 스크린리더가 건너뛰게 하려는 의도적 표시입니다. 이 프로젝트에는 장식 이미지가 없어 전부 채워져 있습니다.

② label for ↔ input id

```
<label for="name">이름</label>
<input type="text" id="name" name="name" autocomplete="name" />
```

속성	역할
<code>for</code>	이 라벨이 어느 입력칸의 이름인지 지정. <code>id</code> 와 정확히 일치 해야 함
<code>id</code>	label과 JavaScript가 찾는 이름. 문서에서 유일
<code>name</code>	폼 전송 시 서버가 받는 키. <code>id</code> 와 별개로 둘 다 필요
<code>autocomplete</code>	브라우저 자동완성 힌트 (사용자 편의)

검증 방법

라벨 글자를 클릭했을 때 입력칸에 커서가 가면 짝이 맞는 것입니다. 안 가면 `for` 와 `id` 가 어긋난 것.

③ aria-* — 눈에 안 보이는 상태를 전달

```
<button type="button" class="nav-toggle"
  aria-label="메뉴 열기" aria-expanded="false">
  <span></span><span></span><span></span></button>
```

- `aria-label` — 버튼 안에 `<span>` 3개뿐이라 읽을 글자가 없습니다. 이름을 직접 줘야 함
- `aria-expanded` — 메뉴가 열렸는지 여부. JS가 클래스와 함께 갱신합니다. 화면 상태와 접근성 정보가 어긋나면 스크린리더 사용자만 잘못된 정보를 받습니다

3-5. button vs a — 무엇으로 만들 것인가

	<code><a></code> 링크	<code><button></code> 버튼
본질	이동한다 (다른 위치·문서로)	동작한다 (페이지 안에서 무언가 실행)
이 프로젝트	“프로젝트 보기”, “연락하기”, “App Store에서 보기”, 내비 메뉴	햄버거, 테마 토글, 맨 위로, 폼 제출, 다시 시도
주의	<code>class="btn"</code> 으로 모양만 버튼처럼	<code>type="button"</code> 명시 필수

`type="button"`을 왜 반드시 쓰는가

`<button>`의 기본 `type`은 `submit`입니다. 폼 안에 들어가는 순간 클릭할 때마다 의도치 않게 폼이 제출됩니다. 실제로 재시도 버튼에도 `type="button"`을 붙인 이유입니다.

3-6. novalidate — 브라우저 검증을 끄는 이유

```
<form id="contact-form" novalidate>
```

브라우저 기본 검증(빨간 말풍선)을 끄는 속성입니다. 안 끄면 내가 만든 검증 로직이 실행되기도 전에 브라우저가 가로챍니다. 직접 검증을 구현하는 것이 과제 요구사항 6번이므로 꺼야 합니다.

3-7. defer — 스크립트 로딩 방식 비교

방식	HTML 파싱	다운로드	실행 시점
<code><script></code> (속성 없음)	멈춤	파싱 중단 후	즉시 → DOM이 아직 없어 <code>querySelector</code> 가 <code>null</code> X
<code>async</code>	실행 때 멈춤	병렬	다운로드 끝나는 대로 → 순서 보장 없음
<code>defer</code>	안 멈춤	병렬	DOM 파싱 완료 후, 작성 순서대로 O
body 맨 끝	안 멈춤	그 시점에	DOM 완성 후 (defer와 비슷하나 다운로드 시작이 늦음)

결론

`defer` 를 쓰면 `DOMContentLoaded` 이벤트로 코드를 감싸지 않아도 된다 → **O**
그래서 `main.js` 는 최상단부터 바로 `querySelector` 를 호출합니다.

3-8. `rel="noopener noreferrer"`

`target="_blank"` 로 새 탭을 열면, 열린 페이지가 `window.opener` 를 통해 **원래 내 페이지를** 조작할 수 있습니다(탭내빙, tabnabbing). `noopener` 가 그 연결을 끊고, `noreferrer` 는 어디서 왔는지 정보를 숨깁니다.

규칙: `target="_blank"` 를 쓰면 항상 붙인다.

4. CSS — 토큰 · 레이아웃 · 반응형

4-1. 리셋 — 왜 맨 앞 세 줄인가

```
* { margin: 0; padding: 0; box-sizing: border-box; }
```

브라우저마다 `h1`, `p`, `ul` 의 기본 여백이 다릅니다. 0으로 초기화해 **내가 준 값만 적용**되게 만듭니다.

box-sizing: border-box 가 핵심

값	width 가 가리키는 것	실제 차지 폭 (width 300 + padding 20 + border 1)
content-box (기본값)	내용물만	342px — padding-border가 바깥에 추가로 붙음
border-box	padding-border 포함	300px

결론

border-box 를 쓰면 width: 100% 인 요소에 padding을 줘도 부모를 빠져나가지 않는다 → **O**
그래서 거의 모든 실무 프로젝트가 * 에 이것을 겁니다.

4-2. 디자인 토큰 — 다크모드가 존재 이유

다크모드를 변수 없이 만들면 색이 쓰이는 40군데를 두 벌씩 써야 합니다. 변수를 쓰면 **선언 블록 하나만** 추가하면 됩니다.

```
:root { /* :root = 문서 최상위 = <html> */
  --color-bg: #ffffff;
  --color-text: #1a1a1a;
  --color-primary: #2563eb;
  --space-2: 1rem;
  --transition: 0.3s ease;
}
[data-theme="dark"] {
  --color-bg: #121212;
  --color-text: #e8e8e8; /* 순백 아님 - 눈부심(halation) 방지 */
  --color-primary: #60a5fa; /* 어두운 배경용으로 더 밝게 */
}
body { background: var(--color-bg); color: var(--color-text); }
```

	하드코딩	CSS 변수
다크모드 추가	전체 재작성 (약 80줄)	선언 블록 1개 (10줄)
색 하나 변경	40군데 찾기	1군데
라이트/다크 어긋남	자주 발생	구조적으로 불가능

왜 <html>(=:root)에 거는가 — 두 가지 이유

- ① **상속 범위** — CSS 변수는 상속되므로, 최상위에 선언하면 문서의 모든 후손이 물려받습니다. <body>에 걸면 <html> 자체는 못 받습니다.
- ② **깜빡임(FOUC) 방지** — <html>은 문서에서 가장 먼저 존재하므로 <head> 단계에서 즉시 테마를 씌울 수 있습니다. <body>였다면 body 파싱이 끝날 때까지 기다려야 해 흰 화면이 번쩍입니다.

결론: 다크모드 토글 대상은 `document.documentElement` 다 → [O](#)

4-3. 명시도(specificity) — dark 블록이 이기는 이유

`<html data-theme="dark">` 일 때 두 선택자가 모두 매치됩니다. 왜 아래쪽이 이길까요?

선택자	명시도 (id, class, type)	등급 근거
<code>:root</code>	(0, 1, 0)	가상클래스 = class 등급
<code>[data-theme="dark"]</code>	(0, 1, 0)	속성 선택자 = class 등급

동점입니다. 명시도가 같으면 나중에 선언된 규칙이 이깁니다 — CSS(Cascading Style Sheets)의 “Cascade = 폭포”라는 이름이 여기서 나옵니다.

결론

`[data-theme="dark"]` 블록을 `:root` 보다 위에 쓰면 다크모드가 동작하지 않는다 → [O](#)
순서에 의존하기 싫으면 `:root[data-theme="dark"]` 로 쓰면 명시도가 (0,2,0)이 되어 위치와 무관하게 이깁니다.

4-4. Flexbox vs Grid — 언제 무엇을

	Flexbox	Grid
차원	1차원 — 가로 한 줄 또는 세로 한 줄	2차원 — 행과 열을 동시에
주도권	콘텐츠 크기가 배치를 결정	레이아웃이 먼저 정해지고 콘텐츠가 들어감
이 프로젝트	<code>nav</code> , <code>.hero-actions</code> , <code>.skill-list</code> , <code>.featured-links</code> , <code>.form-field</code>	<code>.projects-grid</code> , <code>.featured-shots</code>

한 줄 판정법: “가로로만 늘어놓으면 되는가?” → Flex. “행과 열이 동시에 필요한가?” → Grid.

내비게이션 — Flexbox

```
nav { display: flex; align-items: center; gap: var(--space-2); }

/* HTML 순서 (로그→햄버거→메뉴→테마)를 화면 순서로 재배치 */
.logo      { order: 1; margin-right: auto; }
.nav-menu  { order: 2; }
.theme-toggle { order: 3; }
.nav-toggle { order: 4; }
```

- `margin-right: auto` — 로고 오른쪽 여백을 무한히 밀어냄 → 로고는 왼쪽 끝, 나머지는 오른쪽 끝 (요구사항 3번)

- `align-items: center` — 교차축(세로) 가운데 정렬
- `order` — HTML 순서를 바꾸지 않고 **시각적 순서만** 재배치

프로젝트 카드 — Grid (한 줄로 반응형 완성)

```
.projects-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: var(--space-3);
}
```

부분	의미
<code>auto-fit</code>	들어갈 수 있는 만큼 열을 자동으로 만든다
<code>minmax(280px, 1fr)</code>	각 열은 최소 280px , 남은 공간은 균등 분배 (1fr)
결과	화면 900px → 3열 · 600px → 2열 · 320px → 1열이 미디어 쿼리 없이 자동

이 한 줄이 요구사항 3번의 “Grid, auto-fit, minmax로 반응형”을 그대로 충족합니다.

카드 개수가 몇 개든, 화면이 몇 px이든 열 수가 알아서 맞춰집니다.

4-5. 모바일 퍼스트

	모바일 퍼스트 (이 프로젝트)	데스크톱 퍼스트
기본 CSS	모바일용	데스크톱용
미디어 쿼리	<code>@media (min-width: ...)</code>	<code>@media (max-width: ...)</code>
사고 방향	좁은 화면 → 넓은 화면으로 추가	넓은 화면 → 좁은 화면으로 제거

이 파일에서 확인할 수 있습니다. `.nav-menu { display: none; }` 이 **기본값**이고, 768px 이상에서 `display: flex` 로 **덮어 씁니다**.

```
/* 기본 = 모바일: 메뉴 숨김, 열리면 헤더 아래 드롭다운 */
.nav-menu { display: none; position: absolute; top: 100%; flex-direction: column; }
.nav-menu.active { display: flex; } /* ← JS가 붙이는 클래스 */

@media (min-width: 768px) { /* 태블릿 이상에서 '추가로' 덮어씌움 */
  .nav-toggle { display: none; } /* 햄버거 숨김 */
  .nav-menu { display: flex; position: static; flex-direction: row; }
  .about-inner { flex-direction: row; } /* 사진 왼쪽, 글 오른쪽 */
}
@media (min-width: 1024px) { .profile-img { width: 220px; } }
```

결론

모바일 퍼스트는 `min-width` 를 쓴다 → **O**

4-6. CSS가 미리 정의해 둔 “상태”들

JS는 아래 클래스를 붙였다 뗀 뿐입니다. 모양은 전부 CSS에 이미 적혀 있습니다.

클래스	누가 붙이나	어떤 모양이 되나
<code>header.scrolled</code>	스크롤 > 60px	배경색 + 그림자 생김
<code>.nav-menu.active</code>	햄버거 클릭	<code>display: none</code> → <code>flex</code>
<code>.to-top.show</code>	스크롤 > 300px	<code>opacity: 0</code> → <code>1</code> , 클릭 가능해짐
<code>.reveal.visible</code>	Intersection Observer	투명·아래 24px → 불투명·제자리 (페이드 인)
<code>input.invalid</code>	폼 검증 실패	테두리 빨강

4-7. display:none 대신 opacity를 쓴 이유

```
.to-top {
  opacity: 0;
  pointer-events: none;          /* 안 보일 때 클릭 차단 */
  transition: opacity var(--transition);
}
.to-top.show { opacity: 1; pointer-events: auto; }
```

	<code>display: none/block</code>	<code>opacity: 0/1</code>
transition	안 먹음 (있다/없다의 이진 상태)	부드럽게 전환됨
공간 차지	차지하지 않음	차지함 (fixed라 무관)
클릭	자동으로 불가	가능해서 <code>pointer-events: none</code> 필요

결론

`display` 는 `transition` 이 적용되지 않는다 → **○**. 부드럽게 나타내려면 `opacity` (또는 `visibility`)를 쓰고, 대신 `pointer-events` 로 클릭을 막아야 한다.

4-8. 자잘하지만 물어보면 답해야 하는 것들

코드	이유
<code>scroll-margin-top: 64px</code>	헤더가 <code>fixed</code> 라 앵커로 점프하면 제목이 헤더 밑에 가립니다 . 스크롤 목적지를 64px 위로 밀어줌
<code>body { padding-top: 64px }</code>	<code>fixed</code> 헤더는 문서 흐름에서 빠져 첫 화면을 덮습니다. 그만큼 여백 확보
<code>clamp(1.75rem, 5vw, 3rem)</code>	화면폭에 따라 글자 크기가 자동 조절. 최소 1.75rem, 기준 5vw, 최대 3rem
<code>object-fit: cover</code>	정사각형 틀에 넣어도 비율이 찌그러지지 않음 . 넘치는 부분만 잘라냄
<code>flex-shrink: 0</code>	화면이 좁아져도 프로필 이미지는 줄어들지 않게 고정
<code>flex-grow: 1</code> (카드 p)	설명이 남는 공간을 채워 카드 높이가 서로 맞춰짐
<code>grid-column: 1 / -1</code>	상대 박스가 그리드의 첫 열부터 마지막 열까지 전체 폭 을 차지
<code>min-height: 1.2em</code> (에러 문구)	문구 자리를 미리 비워둠 → 에러가 떠도 아래 요소가 밀리지 않음 (레이아웃 시프트 방지)
<code>font-family: inherit</code> (input)	폼 요소는 body 폰트를 상속받지 않는 예외. 명시해야 통일됨
<code>prefers-reduced-motion</code>	OS에서 “동작 줄이기”를 켜 사용자에게 애니메이션을 최소화 (접근성)

실제로 겪은 실수 ② — 타입 선택자의 사정거리

Featured 카드 안에 `<header class="featured-head">` 를 썼는데, CSS에 이런 규칙이 있었습니다.

```
header { position: fixed; top: 0; width: 100%; }
```

`header` 는 타입 선택자라 문서 안의 모든 `<header>` 에 적용됩니다. 그래서 카드 안의 제목·태그라인이 **화면 최상단에 붕 떠버렸** 습니다.

수정: ① `header {` → `body > header {` (자식 결합자로 범위 한정) ② 카드 안은 `<div>` 로 교체.

결론: 레이아웃 스타일을 시맨틱 태그 이름에 직접 걸면 문서 어디에 있는 같은 태그든 전부 걸린다 → `nav` , `footer` , `section` 도 같은 함정이 있습니다.

실제로 겪은 실수 ③ — CSS는 오타를 조용히 무시한다

`backgroud` , `nnone` , `--max-widht` , `.about-innter` 같은 오타는 **에러를 내지 않고 그 선언만 버려집니다**.

	JavaScript	CSS
모르는 이름을 만나면	<code>ReferenceError</code> 던지고 중단	그 줄만 무시하고 계속

결론: “분명 썼는데 스타일이 안 먹는다”면 오타부터 의심한다 → `O`

확인법: Chrome DevTools → Elements → Styles 패널에서 **취소선이 그어진 속성**이 무시된 선언입니다.

5. JavaScript — 9개 블록 정독

`main.js` 는 0~9번 블록으로 나뉘어 있습니다. 각 블록이 무엇을 감지해서 무엇을 바꾸는지만 보면 됩니다.

블록 0 — 설정값

```
const NAV_SCROLL_THRESHOLD = 60;    // 헤더 배경이 바뀌는 위치
const TOP_BTN_THRESHOLD    = 300;    // 맨 위로 버튼이 나타나는 위치
const REVEAL_THRESHOLD     = 0.2;    // Observer 임계값
const THEME_KEY             = 'portfolio-theme';
```

왜 상수로 뽑았나: ① 숫자가 코드 중간에 흩어져 있으면 무슨 뜻인지 알 수 없다(매직 넘버). ② 값을 바꿀 때 한 곳만 고치면 된다.
③ 이 세 값은 README에 명시해야 하는 기준값입니다(과제 요구사항).

블록 1 — DOM 선택

```
const root      = document.documentElement;    // <html>
const header    = document.querySelector('body > header');
const navLinks  = document.querySelectorAll('.nav-menu a');
const sections  = document.querySelectorAll('main > section');
```

메서드	반환	비고
<code>querySelector(sel)</code>	맞는 것 중 첫 번째 1개	없으면 <code>null</code>
<code>querySelectorAll(sel)</code>	맞는 것 전부 (NodeList)	없으면 빈 목록. <code>forEach</code> 사용 가능

- 인자는 **CSS 선택자 문자열** — CSS에서 쓰던 문법을 그대로 씁니다
- `'body > header'` 처럼 범위를 좁힌 것은 4-8의 실수 ②를 겪은 뒤 명확히 한 것
- 맨 위에서 한 번만 찾아 변수에 담습니다. 이벤트마다 다시 찾으면 그때마다 DOM을 훑어 낭비입니다

블록 2 — 햄버거 메뉴

```
navToggle.addEventListener('click', () => {
  const isOpen = navMenu.classList.toggle('active');

  navToggle.setAttribute('aria-expanded', String(isOpen));
  navToggle.setAttribute('aria-label', isOpen ? '메뉴 닫기' : '메뉴 열기');
});
```

메서드	동작
<code>classList.add('x')</code>	붙인다 (이미 있으면 그대로)
<code>classList.remove('x')</code>	뺀다
<code>classList.toggle('x')</code>	있으면 떼고 없으면 붙인다. 반환값 = 지금 붙어있는지 (true/false)
<code>classList.toggle('x', 조건)</code>	조건이 true면 붙이고 false면 뺀다 (블록 4에서 사용)

여기서 배울 점

`toggle` 의 **반환값**을 받아서 접근성 속성까지 같이 갱신합니다. 화면(클래스)과 보조기술 정보(`aria-expanded`)가 어긋나면 스크린리더 사용자만 잘못된 정보를 받습니다. 상태를 바꿀 때는 그 상태를 표현하는 모든 곳을 함께 갱신해야 합니다.

블록 3 — 부드러운 스크롤

```
navLinks.forEach((link) => {
  link.addEventListener('click', (event) => {
    const targetId = link.getAttribute('href'); // "#about"
    const target = document.querySelector(targetId);
    if (!target) return; // 방어 코드

    event.preventDefault(); // 기본 점프 차단
    target.scrollIntoView({ behavior: 'smooth' });
    closeMenu(); // 모바일에서 메뉴 닫기
  });
});
```

- `event.preventDefault()` — 브라우저의 **기본 동작**(앵커로 순간이동)을 막습니다. 막지 않으면 두 동작이 겹칩니다
- `scrollIntoView({behavior:'smooth'})` — 해당 요소가 보이도록 **부드럽게** 스크롤
- `if (!target) return` — 대응하는 섹션이 없으면 아무것도 하지 않습니다. `null` 에 메서드를 호출하면 `TypeError` 로 그 아래 모든 코드가 멈춥니다
- 메뉴를 누르면 닫히게 한 것은 **모바일 UX** — 안 닫으면 이동 후에도 메뉴가 화면을 덮습니다

참고: CSS의 `html { scroll-behavior: smooth }` 만으로도 부드러운 스크롤은 됩니다. JS로도 구현한 이유는 **메뉴 닫기를 같은 클릭에서 처리하기** 위해서입니다.

블록 4 — 스크롤 이벤트 (리스너 1개로 기능 2개)

```
const handleScroll = () => {
  const y = window.scrollY; // 현재 세로 스크롤 위치(px)

  header.classList.toggle('scrolled', y > NAV_SCROLL_THRESHOLD);
  toTopBtn.classList.toggle('show', y > TOP_BTN_THRESHOLD);
};

window.addEventListener('scroll', handleScroll, { passive: true });
handleScroll(); // 초기 1회 실행
```

포인트	이유
리스너 1개 로 두 기능	<code>scroll</code> 은 1초에 수십 번 발생합니다. 리스너를 늘리면 그만큼 비용이 커집니다
<code>toggle(클래스, 조건)</code>	<code>if/else</code> 로 <code>add/remove</code> 를 나눠 쓰지 않아도 됩니다
<code>{ passive: true }</code>	“이 리스너는 <code>preventDefault</code> 를 쓰지 않겠다”는 약속 . 브라우저가 리스너 결과를 기다리지 않고 즉시 스크롤을 그려 부드러워집니다
마지막 <code>handleScroll()</code>	새로고침 시 이미 페이지 중간에 있을 수 있습니다. 초기 상태를 한 번 맞춰줍니다

블록 5·6 — 맨 위로 버튼 / 다크 모드

```

toTopBtn.addEventListener('click', () => {
  window.scrollTo({ top: 0, behavior: 'smooth' });
});

/* ---- 다크 모드 ---- */
const applyTheme = (theme) => {
  root.setAttribute('data-theme', theme); // ① 상태를 DOM에
  themeToggle.textContent = theme === 'dark' ? 'light mode' : 'dark mode';
  themeToggle.setAttribute('aria-label', ...);
};

applyTheme(root.getAttribute('data-theme') || 'light'); // 진입 시 버튼 문구 동기화

themeToggle.addEventListener('click', () => {
  const next = root.getAttribute('data-theme') === 'dark' ? 'light' : 'dark';
  applyTheme(next);
  localStorage.setItem(THEME_KEY, next); // ② 새로고침 후에도 유지
});

```

[클릭] → next 계산 → `applyTheme(next)` → `<html data-theme>` 변경 → CSS 변수 교체 → 전체 화면 갱신
 ↳ `localStorage.setItem` → 새로고침해도 유지

[재진입] head 인라인 스크립트가 `localStorage`를 읽어 `<html>`에 즉시 적용 (깜빡임 없음)
 → `main.js`의 `applyTheme` 이 버튼 문구만 맞춤

API	설명
<code>localStorage.setItem(k, v)</code>	브라우저에 영구 저장. 탭을 달아도 남음 (<code>sessionStorage</code> 는 탭 닫으면 사라짐)
<code>localStorage.getItem(k)</code>	읽기. 없으면 <code>null</code>
저장 형식	문자열만 저장됩니다. 객체는 <code>JSON.stringify</code> 가 필요
범위	같은 출처(origin) 에서만 공유. 다른 브라우저·기기와는 공유되지 않음

함수로 뺀 이유 (`applyTheme`)

테마 적용은 두 곳에서 필요합니다 — ① 페이지 진입 시, ② 버튼 클릭 시. 같은 로직을 두 번 쓰면 나중에 한쪽만 고치는 사고가 납니다. 함수로 묶으면 **진실의 출처가 하나**가 됩니다.

블록 7 — Intersection Observer

```
sections.forEach((section) => section.classList.add('reveal'));

const observer = new IntersectionObserver(
  (entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) { // 화면 안에 들어왔는가
        entry.target.classList.add('visible');
        observer.unobserve(entry.target); // 한 번 나타나면 감시 해제
      }
    });
  },
  { threshold: REVEAL_THRESHOLD } // 0.2 = 20% 보이면 발동
);

sections.forEach((section) => observer.observe(section));
```

개념	설명
Intersection Observer	“이 요소가 화면에 들어왔는가”를 브라우저가 대신 감시해 주는 API
<code>threshold: 0.2</code>	요소의 20% 가 보이면 콜백 실행. 0 = 1px이라도 보이면, 1 = 전부 보여야
<code>entry.isIntersecting</code>	지금 교차(화면 안)하고 있는지 <code>true/false</code>
<code>observer.unobserve()</code>	감시 중단. 안 하면 스크롤할 때마다 콜백이 계속 돌아 낭비

왜 scroll 이벤트를 안 쓰는가

`scroll` 로 하려면 매 프레임 `getBoundingClientRect()` 로 모든 요소의 위치를 직접 계산해야 합니다. 이는 강제 리플로우 (layout)를 유발해 무겁습니다. Observer는 브라우저가 내부적으로 효율적으로 처리하고, 조건이 맞을 때만 콜백을 부릅니다 → **○**

블록 8 — GitHub API (가장 배점이 큰 부분)

8-4. 호출 본체

```
const loadProjects = async () => {
  renderLoading(); // 상태 ① 로딩

  try {
    const response = await fetch(API_URL);

    if (response.status === 403) {
      throw new Error('GitHub API 요청 한도(시간당 60회)를 초과했습니다...');
    }
    if (!response.ok) {
      throw new Error(`서버 응답 오류 (HTTP ${response.status})`);
    }

    const data = await response.json();
    const repos = data.filter((repo) => !repo.fork && !FEATURED_REPOS.includes(repo.name));

    if (repos.length === 0) { renderEmpty(); return; } // 상태 ④ 빈 데이터

    projectsList.innerHTML = repos.map(createCard).join(''); // 상태 ② 성공
  } catch (error) {
    console.error('[Projects]', error);
    renderError(error.message); // 상태 ③ 에러
  }
};
```

가장 많이 틀리는 지점 — fetch는 HTTP 에러에서 reject하지 않는다

fetch 는 네트워크 자체가 실패했을 때만 reject합니다. 404·403·500 같은 응답은 “정상적으로 받았다”고 보고 resolve합니다. 그래서 try/catch 만 믿으면 에러가 잡히지 않고, 서버가 보낸 에러 페이지를 데이터로 처리하게 됩니다.

반드시 **response.ok** (또는 **status**)를 직접 확인해야 한다 → [O](#)

문법	설명
<code>async</code>	이 함수가 비동기임을 표시. 항상 Promise를 반환
<code>await</code>	Promise가 끝날 때까지 이 함수 안에서만 기다림. 페이지 전체는 멈추지 않음
<code>try/catch</code>	<code>await</code> 가 던진 에러와 <code>throw</code> 한 에러를 한곳에서 처리
<code>response.ok</code>	상태 코드가 200~299면 <code>true</code>
<code>.json()</code>	응답 본문을 JSON으로 파싱. 이것도 비동기라 <code>await</code> 필요

8-3. 카드 생성 — 구조분해 + 템플릿 리터럴 + map

```
const createCard = ({ name, description, html_url, language, stargazers_count, forks_count }) => `

### ${escapeHtml(name)}



${description ? escapeHtml(description) : '설명이 없습니다.'}



...

`;

projectsList.innerHTML = repos.map(createCard).join('');
```

- **구조분해 할당** — 매개변수 자리에서 바로 필요한 값만 꺼냅니다. `repo.name`, `repo.language` ... 를 매번 쓸 필요가 없습니다
- **템플릿 리터럴** — 백틱(```) 문자열. 여러 줄과 `${...}` 값 삽입이 가능해 HTML 조립에 적합합니다
- **map** — 배열의 각 원소를 **다른 것으로 변환해 새 배열**을 만듭니다 (저장소 객체 → HTML 문자열)
- **join('')** — 문자열 배열을 하나로 이어붙입니다. 안 쓰면 원소 사이에 쉼표가 들어갑니다

innerHTML 한 번 vs 반복 append

반복문 안에서 `innerHTML +=` 를 쓰면 매번 **전체를 다시 파싱**합니다(N번 리렌더). `map` 으로 문자열을 다 만든 뒤 **마지막에 한 번** 대입하면 파싱이 1회로 끝납니다 → 

8-1. XSS 방어 — escapeHtml

```
const escapeHtml = (value) =>
String(value).replace(/&|<|>|\"/g, (char) => ({
  '&': '&amp;', '<': '&lt;', '>': '&gt;',
  '\"': '&quot;', '\"': '&#39;',
}[char]));
```

저장소 이름·설명 = 외부에서 온 문자열입니다. `innerHTML` 에 그대로 넣으면 설명에 `<img src=x onerror=...>` 같은 것이 들어 있을 때 **내 페이지에서 실행**됩니다(XSS). HTML에서 의미를 갖는 5개 문자를 **무해한 엔티티**로 바꾸면 글자만으로 표시됩니다.

지금은 내 저장소만 불러오지만, 사용자 입력이나 남의 데이터를 `innerHTML` 에 넣을 때는 **항상** 필요한 처리입니다.

`textContent` 를 쓰면 자동으로 안전하지만, HTML 구조를 만들어야 할 때는 이 방법을 씁니다.

8-5. 재시도 버튼 — 이벤트 위임

```
projectsList.addEventListener('click', (event) => {
  if (event.target.closest('#retry-btn')) {
    loadProjects();
  }
});
```

왜 버튼에 직접 리스너를 못 거는가

재시도 버튼은 **에러가 났을 때 JS가 만들어내는** 요소입니다. 페이지 로드 시점에는 **존재하지 않으므로**

`querySelector('#retry-btn')` 가 `null` 입니다.

그래서 **항상 존재하는 부모**(`projectsList`)에 리스너를 걸고, 클릭이 발생했을 때 **무엇이 클릭됐는지** 확인합니다. 이것을 **이벤트 위임(event delegation)**이라 합니다. 가능한 이유는 이벤트가 자식 → 부모로 **버블링**되기 때문입니다.

`closest('#retry-btn')` 은 클릭된 요소부터 **위로 올라가며** 조건에 맞는 조상을 찾습니다. 버튼 안에 아이콘 같은 자식이 있어도 정확히 잡힙니다(`event.target` 이 아이콘일 수 있으므로).

블록 9 — 폼 유효성 검사

```
const EMAIL_PATTERN = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

const fields = [
  { input: ..., error: ..., validate: (v) => (v.trim() === '' ? '이름을 입력해 주세요.' : '') },
  { input: ..., error: ..., validate: (v) => { /* 빈값 → 형식 순으로 검사 */ } },
  { input: ..., error: ..., validate: ... },
];

const validateField = (field) => {
  const message = field.validate(field.input.value);
  field.error.textContent = message; // 상태 → 화면
  field.input.classList.toggle('invalid', message !== '');
  return message === ''; // 통과 여부
};
```

정규표현식 뜯어보기

조각	의미
<code>^ ... \$</code>	문자열의 처음과 끝 . 이게 없으면 문자열 일부만 맞아도 통과해 버립니다
<code>[^\s@]+</code>	공백(<code>\s</code>)도 <code>@</code> 도 아닌 문자가 1개 이상
<code>@</code>	<code>@</code> 문자 그대로
<code>\.</code>	점 문자 그대로. <code>.</code> 만 쓰면 “아무 문자”라는 뜻이라 이스케이프 필수

→ 요약: **공백·@ 없는 덩어리 @ 덩어리 . 덩어리**. 실무의 완전한 이메일 검증은 훨씬 복잡하지만, 오타를 걸러내는 데는 이 정도가 적절합니다.

제출 핸들러

```
contactForm.addEventListener('submit', (event) => {
  event.preventDefault(); // 새로그침 차단
  formSuccess.textContent = '';

  const results = fields.map((field) => validateField(field));

  if (!results.every(Boolean)) {
    const firstInvalid = fields.find((f) => f.input.classList.contains('invalid'));
    if (firstInvalid) firstInvalid.input.focus(); // 첫 에러로 커서 이동
    return;
  }

  formSuccess.textContent = '메시지가 정상적으로 접수되었습니다. 감사합니다!';
  contactForm.reset();
});
```

설계 판단	이유
<code>map</code> 으로 전부 검사	<code>every / some</code> 을 바로 쓰면 중간에 끊겨 뒤 필드의 에러 메시지가 안 뜹니다. 먼저 전부 검사하고 결과를 모읍니다
<code>every(Boolean)</code>	배열이 전부 true 인지. <code>Boolean</code> 을 함수로 넘겨 각 원소를 참/거짓으로 판정
<code>find(...)</code> + <code>focus()</code>	첫 번째 에러 필드로 커서를 옮겨 사용자가 바로 고칠 수 있게 (UX·접근성)
<code>input</code> 이벤트는 이미 에러인 필드만 재검사	처음 타이핑하는 도중부터 빨간 글씨가 뜨면 불쾌합니다. 한 번 틀린 뒤에는 고치는 즉시 사라지게
<code>trim()</code>	공백만 입력한 것을 빈 값으로 취급
<code>form.reset()</code>	제출 성공 후 입력칸 비우기

실제 전송이 없는 이유: 백엔드 서버가 없기 때문입니다. 과제 요구사항도 “`preventDefault` + 성공 메시지 표시”까지입니다. 실제 전송이 필요하면 보너스 과제의 Formspree·EmailJS를 붙이면 됩니다.

6. ES6+ 문법 총정리

6-1. var를 쓰지 않는 이유

	var	let	const
범위	함수 스코프	블록 스코프 {}	블록 스코프 {}
재선언	가능 (덮어써도 조용함)	불가	불가
재할당	가능	가능	불가
호이스팅	undefined 로 끌어올려짐	TDZ — 선언 전 접근 시 에러	

기본은 `const`, 재할당이 필요할 때만 `let`. `const` 는 “재할당 금지”이지 “내용 불변”이 아닙니다 — `const arr = []` 에 `arr.push()` 는 됩니다.

6-2. 화살표 함수

```
// 일반 함수
function add(a, b) { return a + b; }

// 화살표 함수 — 짧고, 한 줄이면 return 생략
const add = (a, b) => a + b;
const closeMenu = () => { /* 인자 없으면 () */ };
```

`this`를 갖지 않는다는 것이 진짜 차이입니다. 화살표 함수 안의 `this` 는 바깥 스코프의 것을 그대로 씁니다. 그래서 콜백에서 `this` 가 엉뚱한 것을 가리키는 고전적 버그가 사라집니다. 이 프로젝트의 모든 콜백이 화살표 함수인 이유입니다.

6-3. 구조분해 할당

```
// 이전 방식
const createCard = (repo) => `... ${repo.name} ... ${repo.language} ...`;

// 구조분해 — 매개변수 자리에서 바로 꺼냄
const createCard = ({ name, language, stargazers_count }) => `... ${name} ...`;
```

객체의 키 이름으로 값을 꺼냅니다. 순서는 무관하고, 없는 키는 `undefined` 가 됩니다.

6-4. 템플릿 리터럴

```
// 따옴표 방식 — 줄바꿈마다 + 로 잇고 이스케이프 지옥
var s = '<h3>' + name + '</h3>';

// 템플릿 리터럴 — 백틱. 여러 줄 + ${} 삽입
const s = `<h3>${name}</h3>`;
```

`${}` 안에는 표현식이면 뭐든 들어갑니다 — 삼항 연산자, 함수 호출 모두 가능. 이 프로젝트에서는 `API_URL` 조립과 카드 HTML 생성에 씁니다.

6-5. 배열 메서드 — 이 프로젝트에서 쓴 6개

메서드	하는 일	반환	이 프로젝트에서
<code>map</code>	각 원소를 변환	같은 길이의 새 배열	저장소 객체 → 카드 HTML / 필드 → 검증 결과
<code>filter</code>	조건에 맞는 것만 걸러냄	더 짧아진 새 배열	fork-Featured 저장소 제외
<code>forEach</code>	각 원소로 무언가 실행	<code>undefined</code>	리스너 걸기, 클래스 추가, observe 등록
<code>every</code>	전부 조건을 만족하는가	<code>true/false</code>	폼 전체 통과 여부
<code>find</code>	조건에 맞는 첫 원소	원소 또는 <code>undefined</code>	첫 에러 필드 찾기
<code>join</code>	배열을 문자열 로 합침	문자열	카드 HTML 배열 → 하나의 문자열

map vs forEach — 자주 나오는 질문

`map` 은 새 배열을 만들어 반환하고, `forEach` 는 반환값이 없습니다(`undefined`). “변환한 결과가 필요하다면 `map` , 그냥 돌기만 하면 `forEach` ” → [O](#)

`forEach` 의 결과를 변수에 담으려 하는 것이 대표적인 실수입니다.

6-6. 기타 표현

표현	의미
<code>a ? b : c</code>	삼항 연산자. <code>a</code> 가 참이면 <code>b</code> , 아니면 <code>c</code>
<code>a b</code>	<code>a</code> 가 falsy면 <code>b</code> . 기본값 지정에 사용 (<code>getAttribute(...)</code> <code>'light'</code>)
<code>!x</code>	부정. <code>if (!target)</code> <code>return</code> = 없으면 중단
<code>arr.includes(v)</code>	배열에 값이 있는지 <code>true/false</code>
<code>String(x)</code>	문자열로 변환. <code>setAttribute</code> 는 문자열을 요구
<code>'use strict'</code>	엄격 모드. 선언 안 한 변수 사용 등을 에러 로 잡아줌

7. 예상 질문 22개와 모범 답안

문서를 덮고 먼저 답해 보세요. 막히는 항목의 번호가 곧 다시 읽어야 할 장입니다.

HTML

Q1. 시맨틱 태그를 왜 쓰나요? div로도 되지 않나요?

동작은 같지만 의미가 전달되지 않습니다. `header` · `nav` · `main` · `footer` 와 제목이 붙은 `section` 은 랜드마크가 되어, 스크린리더 사용자가 영역 목록을 띄워 원하는 곳으로 바로 이동할 수 있습니다. 검색엔진도 문서 구조를 파악합니다. `div` 는 그 중 아무것도 제공하지 않습니다.

Q2. 구조를 어떤 기준으로 설계했나요?

“제목에 붙일 수 있는 주제 단위”는 `section` , “떼어내도 혼자 말이 되는 콘텐츠”는 `article` , “의미 없이 스타일링만을 위한 상자”는 `div` 로 나눴습니다. 그래서 저장소 카드와 코인 레벨 카드는 `article` , 각 섹션은 `section` , `.hero-actions` · `.form-field` 같은 래퍼는 `div` 입니다.

Q3. section과 article의 차이를 한 문장으로?

RSS 피드에 그것 하나만 떼서 넣어도 말이 되면 `article` , 문서 안의 한 챕터로서만 의미가 있으면 `section` 입니다.

Q4. defer를 왜 붙였나요? async와 뭐가 다른가요?

둘 다 HTML 파싱을 멈추지 않고 병렬로 다운로드하지만, `async` 는 다운로드가 끝나는 대로 실행되어 순서가 보장되지 않고 DOM이 아직 없을 수 있습니다. `defer` 는 DOM 파싱이 끝난 뒤 작성 순서대로 실행됩니다. 그래서 `DOMContentLoaded` 로 감싸지 않고 최상단에서 바로 `querySelector` 를 호출할 수 있습니다.

Q5. head에 인라인 script를 하나 둔 이유는?

다크모드 복원 때문입니다. `main.js` 는 `defer` 라 DOM 파싱 후에야 실행되므로, 거기서 테마를 씌우면 흰 화면이 한 번 번쩍 인 뒤 어두워집니다(FOUC). 그 5줄만 `<head>` 에 인라인으로 두어 CSS가 처음 그려지기 전에 `<html>` 에 테마를 확정합니다.

Q6. label의 for와 input의 id는 왜 맞춰야 하나요?

둘을 연결해야 라벨을 클릭해도 입력칸에 커서가 가고, 스크린리더가 “이름, 편집창”처럼 라벨과 입력을 묶어 읽어줍니다. `id` 는 `label`-JS가 찾는 이름, `name` 은 폼 전송 시 서버가 받는 키라서 둘 다 필요합니다.

Q7. 폼에 novalidate를 붙인 이유는?

브라우저 기본 검증(빨간 말풍선)이 먼저 가로채면 직접 만든 검증 로직이 실행되지 않습니다. 검증을 직접 구현하는 것이 과제 요구사항이므로 껴줍니다.

Q8. Flexbox와 Grid의 차이, 언제 각각을 쓰나요?

Flexbox는 **1차원**(한 줄) 배치로 콘텐츠 크기가 배치를 결정하고, Grid는 **2차원**(행과 열)으로 레이아웃이 먼저 정해집니다. 이 프로젝트에서는 내비게이션·버튼 묶음·기술 태그처럼 한 줄에 늘어놓는 것은 Flexbox, 프로젝트 카드 목록과 앱 스크린샷 갤러리처럼 행과 열이 동시에 필요한 것은 Grid를 썼습니다.

Q9. 프로젝트 카드는 미디어 쿼리 없이 어떻게 반응형이 되나요?

`grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))` 한 줄입니다. `auto-fit` 이 들어갈 수 있는 만큼 열을 자동 생성하고, `minmax(280px, 1fr)` 가 각 열의 최소 폭을 280px로 두되 남은 공간은 균등 분배합니다. 그래서 화면 폭에 따라 3열 → 2열 → 1열이 자동으로 바뀝니다.

Q10. 모바일 퍼스트로 작성했다는 게 코드에서 어디에 드러나나요?

기본 스타일이 모바일용이고, 미디어 쿼리가 전부 `min-width` 입니다. 예를 들어 `.nav-menu { display: none }` 이 기본 값이고 768px 이상에서 `display: flex` 로 덮어씁니다. 데스크톱 퍼스트였다면 반대로 `max-width` 로 좁은 화면에서 제거했을 것입니다.

Q11. 다크모드를 CSS 변수로 만든 이유는?

색을 값이 아니라 **이름**으로 쓰기 때문에, 테마를 바꿀 때 색을 쓰는 모든 규칙을 고칠 필요 없이 **변수 선언 블록 하나만** 교체하면 됩니다. 하드코딩이었다면 색이 쓰인 40여 군데를 두 벌로 유지해야 하고, 한쪽만 고치면 라이트/다크가 어긋납니다.

Q12. `[data-theme="dark"]` 가 `:root` 를 어떻게 이기나요?

이기는 게 아니라 **동점**입니다. `:root` 는 가상클래스, `[data-theme]` 는 속성 선택자로 둘 다 명시도 (0,1,0)입니다. 명시도가 같으면 **나중에 선언된 것이** 적용되므로, dark 블록을 아래에 두어야 합니다. 순서에 의존하지 않으려면 `:root[data-theme="dark"]` 로 쓰면 (0,2,0)이 되어 위치와 무관합니다.

Q13. 다크모드 속성을 왜 body가 아니라 html에 거나요?

두 가지 이유입니다. ① CSS 변수는 상속되므로 최상위인 `<html>` (`:root`)에 걸어야 문서 전체가 물려받습니다. ② `<html>` 은 가장 먼저 존재하므로 `<head>` 단계에서 즉시 테마를 씌워 **깜빡임을 없앨 수** 있습니다.

Q14. 맨 위로 버튼에 `display:none` 대신 `opacity`를 쓴 이유는?

`display` 는 **transition이 적용되지 않아** 버튼이 푹 나타납니다. `opacity` 는 부드럽게 전환되지만 보이지 않아도 클릭이 되므로, `pointer-events: none` 으로 클릭을 함께 막았습니다.

Q15. `querySelector`로 요소를 잡고 `addEventListener`를 거는 흐름을 설명해 보세요.

① `querySelector`에 CSS 선택자를 넘겨 DOM에서 요소 객체를 찾아 변수에 담습니다. ② 그 요소에 `addEventListener('click', 콜백)`으로 “이 이벤트가 나면 이 함수를 실행하라”고 등록합니다. ③ 사용자가 클릭하면 브라우저가 그 콜백을 호출하고, 콜백 안에서 `classList` · `textContent` · `setAttribute`로 DOM을 바꿉니다. ④ 바뀐 DOM에 맞춰 CSS가 화면을 다시 그립니다.

Q16. HTML의 `onclick` 대신 `addEventListener`를 쓰는 이유는?

① 구조(HTML)와 동작(JS)이 분리되어 유지보수가 쉽습니다. ② `onclick`은 **하나만** 등록되어 나중 것이 앞의 것을 덮어쓰지만 `addEventListener`는 여러 개를 겹칠 수 있습니다. ③ `{ passive: true }` 같은 옵션을 줄 수 있습니다. ④ 과제 제약 사항이기도 합니다.

Q17. 스크롤 리스너에 `passive: true`를 왜 붙였나요?

“이 리스너는 `preventDefault`를 호출하지 않겠다”는 약속입니다. 이게 없으면 브라우저는 리스너가 스크롤을 취소할지 몰라 **실행이 끝날 때까지 기다렸다가** 화면을 그립니다. 붙이면 즉시 그릴 수 있어 스크롤이 부드러워집니다.

Q18. `fetch`와 `async/await`로 데이터를 가져오는 흐름과, 세 가지 상태를 어떻게 표현했나요?

함수 시작에서 바로 `renderLoading()`으로 스피너를 띄웁니다. `await fetch(...)`로 응답을 기다리고, `response.ok`를 확인해 실패면 `throw`합니다. 성공하면 `await response.json()`으로 파싱하고, `filter`로 `fork`를 걸러낸 뒤 `map`으로 카드 HTML을 만들어 `innerHTML`에 한 번에 넣습니다. 배열이 비어 있으면 빈 상태를, `catch`에서는 에러 메시지와 재시도 버튼을 렌더링합니다.

Q19. `fetch`는 404가 나면 `catch`로 가나요?

가지 않습니다. `fetch`는 네트워크 자체가 실패했을 때만 `reject`합니다. 404·403·500은 “응답을 정상적으로 받았다”고 보고 `resolve`하므로, `response.ok`나 `status`를 **직접 확인**해 `throw`해야 `catch`로 넘어갑니다. 이 프로젝트에서는 403(레이트 리밋)을 따로 잡아 안내 문구를 다르게 줍니다.

Q20. 재시도 버튼에는 왜 직접 리스너를 걸지 않았나요?

그 버튼은 **에러가 났을 때 JS가 만들어내는** 요소라 페이지 로드 시점에는 존재하지 않습니다. 그래서 항상 존재하는 부모에 리스너를 걸고, 클릭이 버블링으로 올라오면 `event.target.closest('#retry-btn')`으로 대상을 확인합니다. 이것을 **이벤트 위임**이라고 합니다.

Q21. `innerHTML`에 API 데이터를 그대로 넣으면 왜 위험한가요?

저장소 설명 같은 외부 문자열에 `<img src=x onerror=...>` 같은 태그가 들어 있으면 **내 페이지에서 실행**됩니다(XSS). 그래서 `&` · `<` · `>` · `"` · `'` 다섯 문자를 HTML 엔티티로 바꾸는 `escapeHtml`을 거쳐 글자만으로 표시되게 했습니다.

Q22. “하나의 기능”이 이벤트 → 상태 → 렌더링으로 이어지는 과정을 예로 설명해 보세요.

다크모드가 가장 명확합니다. ① **이벤트**: 토글 버튼 클릭. ② **상태 변경**: JS가 `<html>` 의 `data-theme` 을 `light` ↔ `dark` 로 바꾸고 `localStorage`에 저장. ③ **렌더링**: CSS가 `[data-theme="dark"]` 블록의 변수로 교체하면서 배경·글자·카드·폼·푸터가 한꺼번에 바뀝니다. 여기서 JS는 **색이 무엇인지조차 모릅니다**. 상태만 바꾸고 표현은 CSS가 전담하는 이 분업이 React의 상태-렌더링 흐름과 같은 구조입니다.

8. 요구사항 대조표 & 자가 점검

8-1. 기능 요구사항 1~10

#	요구사항	충족	코드 위치
1	폴더 구조 분리 · 외부 CSS/JS 연결 · Live Server	O	index.html / css/ / js/ / images/ , link + script defer
2	시맨틱 마크업 · 6개 섹션 · 앵커 · alt · label	O	index.html 전체. header/nav/main/section/article/footer
3	CSS 변수 · 다크 토큰 · Flex 내비 · Grid 카드 · 모바일 퍼스트 · hover/transition/shadow	O	style.css 2~3장(토큰), 5장(nav), 10장(grid), 15장(미디어 쿼리)
4	defer · const/let만 · addEventListener · querySelector · textContent/innerHTML · classList · click/submit/scroll/input · preventDefault	O	main.js 전체 블록
5	인터랙션 5종 (햄버거 · 부드러운 스크롤 · 스크롤 탑 · 내비 스타일 · 다크모드 · 스크롤 애니메이션)	O	main.js 블록 2·3·4·5·6·7
6	폼 UX — 필수값 · 이메일 형식 · 필드 근처 에러 · preventDefault · 성공 메시지	O	main.js 블록 9, .error-msg / .success-msg
7	화살표 함수 · 템플릿 리터럴 · 구조분해 · map/filter/forEach	O	createCard (구조분해+리터럴), repos.map , data.filter , sections.forEach
8	fetch + async/await · 로딩/성공/에러/빈 상태 · try/catch · 403 처리	O	main.js 블록 8 (loadProjects)
9	“상태 → 렌더링” 흐름 3가지 이상	O	5가지 확보 — 2장 표 참고
10	GitHub Pages 배포 · README(설명·기술·URL·스크린샷)	O	dnsxkm.github.io/portfolio , README.md

8-2. 제약사항 점검

제약	준수	확인 방법
외부 라이브러리 금지 (React/Vue/jQuery/Bootstrap/Tailwind)	○	index.html 에 외부 script ·CDN 없음
var 미사용	○	main.js 에 var 0개
인라인 onclick 미사용	○	모든 이벤트가 addEventListener
인라인 style="..." 미사용	○	index.html 에 style= 0개

8-3. README에 명시해야 하는 기준값

항목	값	정의 위치
스크롤 탑 버튼 등장 기준	300px	TOP_BTN_THRESHOLD
내비게이션 배경 변경 기준	60px	NAV_SCROLL_THRESHOLD
Intersection Observer 임계값	0.2	REVEAL_THRESHOLD
localStorage 키	portfolio-theme	THEME_KEY

8-4. 발표 전 동작 점검 체크리스트

- ☐ 창을 768px 아래로 좁히면 햄버거가 나타나고 메뉴가 사라진다
- ☐ 햄버거를 누르면 메뉴가 열리고, 메뉴 항목을 누르면 이동 후 **자동으로 닫힌다**
- ☐ 내비게이션 클릭 시 부드럽게 스크롤되고, 섹션 제목이 헤더에 **가리지 않는다**
- ☐ 60px 이상 스크롤하면 헤더에 배경색과 그림자가 생긴다
- ☐ 300px 이상 스크롤하면 우하단에 원형 버튼이 **서서히** 나타난다
- ☐ 다크모드 토글 → **새로고침(⌂R)** → 어두운 상태가 유지된다 (**요구사항 4의 핵심**)
- ☐ 다크모드에서 카드·폼·푸터·에러 문구까지 **전부** 어두운 테마다
- ☐ 스크롤을 내리면 각 섹션이 아래에서 위로 페이드 인된다
- ☐ Projects에 저장소 카드가 뜬다 (또는 빈/에러 상태가 정상 표시된다)
- ☐ 빈 폼을 제출하면 **세 필드 모두** 에러가 뜨고 첫 필드로 커서가 간다
- ☐ abc 만 입력하고 제출하면 “이메일 형식이 올바르지 않습니다”가 뜬다
- ☐ 에러 상태에서 글자를 입력하면 **즉시** 에러가 사라진다
- ☐ 정상 입력 후 제출하면 초록색 성공 메시지가 뜨고 입력칸이 비워진다
- ☐ 라벨 글자를 클릭하면 해당 입력칸에 커서가 간다
- ☐ F12 Console에 **빨간 에러가 하나도 없다**
- ☐ 배포된 URL(dnsxkm.github.io/portfolio)에서 위 항목이 모두 동작한다

8-5. 아직 남아 있는 개선 여지 (정직하게 말할 것)

항목	현황과 이유
폼 실제 전송	백엔드가 없어 성공 메시지만 표시합니다. 실제 전송은 Formspree·EmailJS를 붙이면 됩니다 (보너스 과제)
프로필 사진	플레이스홀더 이미지입니다. 같은 파일명으로 덮어쓰면 코드 수정 없이 교체됩니다
코인 레벨 기술 스택	App Store에서 확인 가능한 정보만 기재했습니다. 프레임워크·백엔드 언어는 직접 추가해야 정확합니다
GitHub API 인증	인증 없이 호출하므로 시간당 60회 제한이 있습니다. 초과 시 403 에러 상태 UI가 뜨도록 처리했습니다
프로젝트 언어별 필터링	보너스 과제라 구현하지 않았습니다. 이미 <code>filter</code> 를 쓰고 있어 확장은 어렵지 않습니다

마지막으로 — 이 문장 하나만 남기세요

JavaScript는 클래스와 속성만 바꾼다. 어떻게 보일지는 CSS가 전담한다.

이 프로젝트의 모든 기능이 이 한 줄의 변주입니다. 그리고 다음 미션인 React의 컴포넌트 · 상태 · 이벤트는 바로 이 구조를 자동화한 것입니다.